

Linux System Call Tracer

A **Real-Time Dynamic Linux System Call Tracer** developed in **C** using Linux System Programming concepts and the `ptrace()` system call.

The tracer monitors a user-specified process and captures its interaction with the Linux kernel in real time. It identifies system calls, tracks their entry and exit points, captures register information, displays executable paths, and dynamically follows newly created processes such as children and grandchildren.

Project Overview

Every application running on Linux interacts with the operating system through **system calls**.

This project provides a low-level mechanism for observing these interactions using the Linux `ptrace()` system call.

The tracer attaches to a target process and intercepts its system calls, allowing information such as:

- System call number
- System call name
- Entry and exit events
- Process ID
- Executable path
- CPU register values
- Process creation and termination
- Parent-child process relationships

to be monitored during execution.

The project is designed using a modular C architecture so that process management, system-call identification, logging, and tracing logic remain separated.

Features

◆ Real-Time System Call Tracing

Uses Linux `ptrace()` to intercept system calls as they are executed.

◆ System Call Entry & Exit Detection

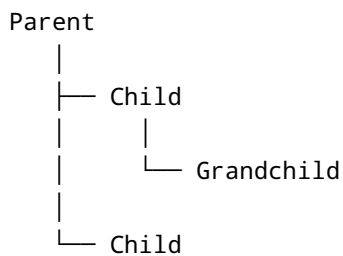
The tracer identifies both:

```
ENTRY
EXIT
```

events for system calls, providing visibility into the complete system-call lifecycle.

◆ **Dynamic Process Tracking**

The tracer can detect and track processes created by the target process, including:



This allows tracing of applications that create additional processes during execution.

◆ **System Call Identification**

System call numbers are mapped to human-readable names.

Example:

```
60 → exit
39 → getpid
1  → write
0  → read
9  → mmap
3  → close
```

◆ **Register Information**

Relevant CPU register values are captured during system-call processing to provide additional low-level information about the process state.

◆ **Executable Path Tracking**

The tracer identifies the executable associated with a traced process.

Example:

Executable Path: /usr/bin/bash

♦ **Dynamic Process Table**

The project maintains an internal process table containing currently tracked processes.

Processes can be dynamically:

ADD
REMOVE

as they are created and terminated.

♦ **Modular Architecture**

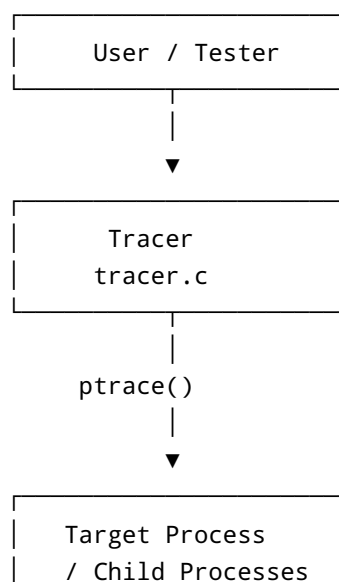
The project is divided into multiple source files, making the code easier to maintain, debug, and extend.

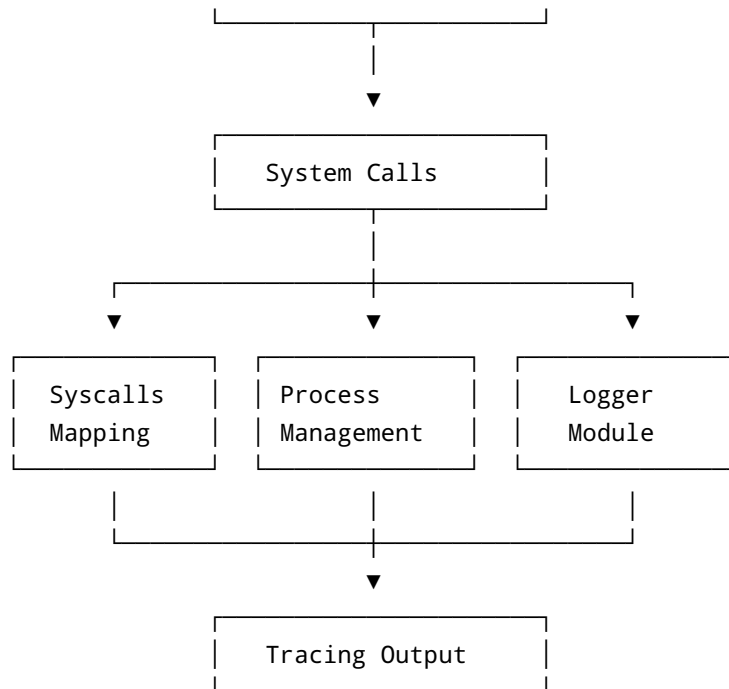
♦ **Structured Logging**

Tracing information can be stored through the dedicated logging module for later analysis.

Architecture

The tracer follows a modular architecture:





Project Structure

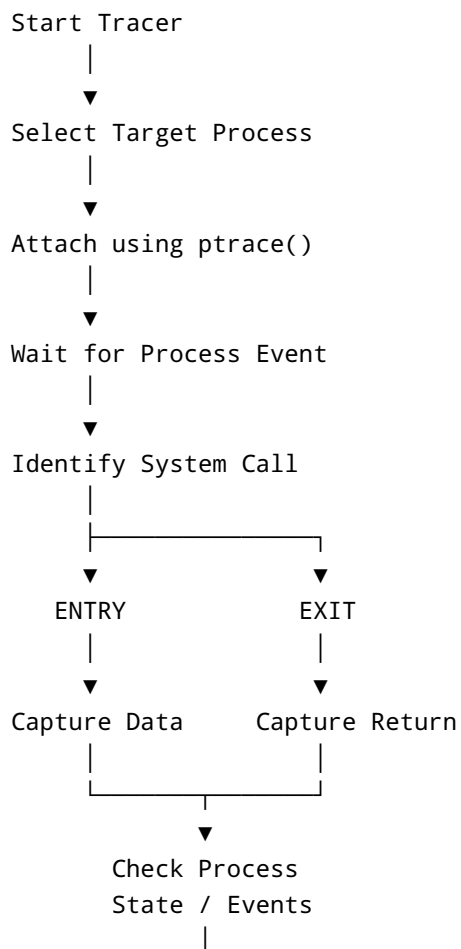
```
Linux-SystemCall-Tracer/  
|  
├── include/  
│   └── logger.h  
|  
├── src/  
│   ├── tracer.c  
│   ├── syscalls.c  
│   ├── process_info.c  
│   ├── process_table.c  
│   └── logger.c  
|  
├── logs/  
|  
├── Testing/  
│   └── test programs  
|  
└── README.md
```

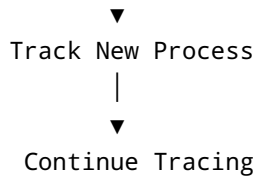
Source Files

File	Responsibility
<code>tracer.c</code>	Main tracing engine and <code>ptrace()</code> control
<code>syscalls.c</code>	System call number-to-name mapping
<code>process_info.c</code>	Process and executable information
<code>process_table.c</code>	Dynamic process tracking
<code>logger.c</code>	Logging and output management
<code>logger.h</code>	Logger declarations and interfaces

How It Works

The basic tracing workflow is:





The tracer repeatedly waits for process events and determines whether the process has entered or exited a system call.

Requirements

The project requires a Linux environment with:

- GCC
- Linux kernel
- POSIX/Linux system-call support
- Standard C development libraries
- `ptrace()` support

Recommended environment:

```
Linux
GCC
GNU Make (optional)
```

Note: `ptrace()` is Linux-specific, so the project is intended to run on Linux rather than native Windows.

Installation

Clone the repository:

```
git clone <YOUR_GITHUB_REPOSITORY_URL>
```

Navigate into the project:

```
cd Linux-SystemCall-Tracer
```

Create the required directories if they do not already exist:

```
mkdir -p logs
```

Compilation

Compile the project using GCC:

```
gcc src/tracer.c src/syscalls.c src/process_info.c src/process_table.c src/
logger.c -Iinclude -o tracer
```

After successful compilation:

```
tracer
```

should be generated in the project directory.

You can verify it using:

```
ls
```

Usage

Run the tracer according to the target-process mode implemented by the project.

For example, if tracing a process by PID:

```
./tracer <PID>
```

Example:

```
./tracer 12345
```

The tracer will attach to the specified process and begin monitoring its system calls.

Testing

A dedicated testing directory can be used for programs designed to exercise different tracing scenarios.

Example test program:

```
Testing/  
└─ fork_test.c
```

A test application can create multiple processes:

```
Parent Process  
  |  
  └─ Child Process  
      |  
      └─ Grandchild Process  
  |  
  └─ Child Process
```

This allows the tracer to be tested against:

- Basic system calls
- Process creation
- Multiple processes
- Process termination
- Parent-child relationships
- Dynamic process tracking
- Longer-running processes

Sample Output

A typical tracing session may produce output similar to:

```
=====
      LINUX SYSTEM CALL TRACER
=====

PID: 24561
Executable Path: /home/user/Testing/fork_test

-----
```


ENTRY

PID : 24561
Syscall : 39
Name : getpid
Executable : /home/user/Testing/fork_test

Registers:

RAX : 0x27
RDI : 0x0
RSI : 0x0
RDX : 0x0

EXIT

PID : 24561
Syscall : 39
Name : getpid
Return : 24561

ENTRY

PID : 24561
Syscall : 56
Name : clone
Executable : /home/user/Testing/fork_test

PROCESS EVENT

New Process Detected
Parent PID : 24561
Child PID : 24562

ENTRY

PID : 24562
Syscall : 1
Name : write
Executable : /home/user/Testing/fork_test

EXIT

PID : 24562

```
Syscall    : 1
Return     : 18
```

The exact output depends on the target program and the system calls executed by it.



Logging

The project contains a dedicated logging module responsible for managing tracing output.

The logging functionality is designed to:

- Create log files
- Store tracing events
- Record process information
- Record system-call information
- Flush output during tracing
- Close log files cleanly

Logs are stored inside:

```
logs/
```

Example:

```
logs/
├─ trace_2026_08_10.log
└─ ...
```



System Calls

The tracer maintains a mapping between Linux system-call numbers and their corresponding names.

Examples include:

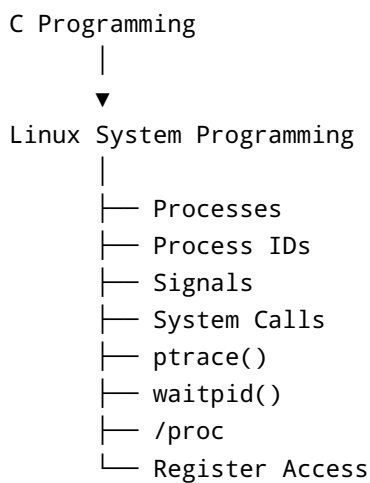
System Call	Purpose
<code>read()</code>	Read data
<code>write()</code>	Write data

System Call	Purpose
<code>open()</code>	Open a file
<code>close()</code>	Close a file descriptor
<code>mmap()</code>	Map memory
<code>munmap()</code>	Unmap memory
<code>fork()</code>	Create a process
<code>clone()</code>	Create a process/thread
<code>execve()</code>	Execute a program
<code>exit()</code>	Terminate a process

The mapping allows numerical syscall information obtained through `ptrace()` to be converted into readable output.

Technologies & Concepts

This project demonstrates practical implementation of:



Core Concepts

- Linux process management
- System-call execution
- User space ↔ kernel interaction
- Process tracing
- Dynamic process tracking

- Signals and process states
 - CPU registers
 - Linux `/proc` filesystem
 - Low-level debugging concepts
-

Why `ptrace()`?

`ptrace()` provides a mechanism for one process to observe and control another process.

For this project, it provides the ability to:

```
Attach
  ↓
Observe
  ↓
Intercept System Calls
  ↓
Inspect Registers
  ↓
Track Process Events
  ↓
Continue Execution
```

This makes it suitable for building a low-level system-call tracing tool without requiring modifications to the target application.

Project Screenshots

Add screenshots of your actual tracer execution here.

Tracer Startup

![Tracer Startup](screenshots/tracer-startup.png)

System Call Entry/Exit

![System Call Trace](screenshots/syscall-trace.png)

Process Tracking

![[Process Tracking](screenshots/process-tracking.png)]

Log Output

![[Log Output](screenshots/log-output.png)]

Recommended screenshot directory:

```
screenshots/  
├─ tracer-startup.png  
├─ syscall-trace.png  
├─ process-tracking.png  
└─ log-output.png
```



Future Enhancements

Possible future improvements include:

- Command-line argument parsing
- More complete syscall coverage
- Improved syscall argument decoding
- Filtering system calls by name
- Filtering by PID
- Improved JSON logging
- Signal-event visualization
- Performance statistics
- Multi-threaded process tracking
- Terminal-based user interface
- Integration with eBPF for high-performance tracing



Learning Outcomes

Through this project, the following concepts were explored:

- How applications interact with the Linux kernel
- How system calls can be intercepted
- How `ptrace()` works

- How processes are created and terminated
 - How parent-child process relationships can be tracked
 - How CPU registers can be inspected
 - How Linux process information can be obtained
 - How a modular system-programming project can be designed in C
-



Author

Aminesh Bajpai

Developed as a Linux System Programming project focused on understanding **process tracing, system calls, and low-level Linux internals**.



Acknowledgements

This project was developed for educational purposes to explore Linux internals, system programming, process management, and debugging mechanisms.

If you found this project useful, consider giving the repository a ★.